CASE STUDY

Agentic AI Logistics Control Tower

Agents 1, 2 & 3 — Last Mile Europe

Germany · France · Netherlands

Author: Sarthak Shirsat

Logistics Operator turned AI Builder | Founder @ Acharooz | Ex CPO Movam| Ex Tolaram Group | IIM Mumbai

May 2026

github.com/sarthakshirsatai-lab/ai-logistics-control-tower

Project Overview

This case study documents the design, logic and simulation results of an Agentic AI Logistics Control Tower — a personal learning project built to demonstrate applied AI product thinking in the context of European last-mile delivery operations.

The system is built as a series of five modular AI agents, each solving a distinct operational problem. This document covers Agents 1, 2 and 3.

Background & Motivation

The author has 8.5 years of senior experience across end-to-end logistics operations, B2B logistics SaaS, and D2C brand building:

- Fleet Manager → Logistics Head → Branch Head at Tolaram Group (Nigeria) — managing fleets of 150+ trucks
- Co-Founder & CPO at Movam Technologies (Nigeria) — built an API-based B2B logistics SaaS platform for 3,000+ trucks
- Founder, Acharooz — D2C premium pickle brand, available on Amazon India
- MBA, IIM Mumbai

Having lived the operational pain of last-mile delivery decisions firsthand — managing failed deliveries, courier performance issues and customer escalations under pressure — the objective was to build AI agents that systematise these decisions, removing gut feel and replacing it with data-driven logic.

System Architecture

The Control Tower is designed as five modular agents with a human-in-the-loop decision layer:

Agent	Name	Function
Agent 1	**Last Mile Exception Detector**	Detects exceptions in real-time shipment data — failed attempts, address errors, customer absent events. Classifies severity and determines AUTO-EXECUTE vs ESCALATE.
Agent 2	**Courier Performance Monitor**	Scores couriers across 30-day rolling data using OTD rate, courier-attributable miss rate and exception rate. Outputs RAG rating with recommended action.
Agent 3	**Exception Resolution Agent**	Given a shipment exception, evaluates all remediation options using a weighted composite score across SLA Recovery, Cost and Customer Experience. Recommends optimal action.
Agent 4	**Customer Communication Agent**	Generates personalised customer messages based on exception type and resolution chosen. Uses Claude API for dynamic language generation. (Pending)

Agent 5	**Orchestrator**	Connects all agents into a single pipeline. Manages AUTO-EXECUTE vs ESCALATE routing. Human-in-the-loop dashboard. (Pending)

Agent 1 — Last Mile Exception Detector

Objective

Detect exceptions in real-time shipment data across multiple couriers and countries. Classify each exception by severity. Determine whether to AUTO-EXECUTE a resolution or ESCALATE to a human dispatcher.

The Problem It Solves

In a last-mile operation handling hundreds of daily shipments, exceptions happen continuously — customer absent, address errors, SLA breach risks. Without a detection system, these surface only when the customer complains. By then the SLA is already missed.

Agent 1 acts as a real-time sensor — scanning shipment data, flagging exceptions the moment they occur and routing them appropriately before the SLA window closes.

Input Data

- Shipment ID, Courier, Country, Postcode
- Delivery status (out_for_delivery, failed_attempt, delivered, etc.)
- SLA deadline and hours remaining
- Exception type detected
- Customer tier flag

Exception Types Detected

Exception	Severity	Typical Action
SLA Breach Risk	HIGH	Notify customer, escalate
Failed Attempt	HIGH	Reroute or re-attempt
Customer Absent	MEDIUM	Parcel locker / PUDO
Address Error	MEDIUM-HIGH	Always escalate — manual fix
Regional Delay	MEDIUM	Monitor, pre-notify
Courier Underperformance	HIGH	Flag to Agent 2

Key Design Decision — Detection Only

Agent 1 acts purely as a real-time sensor. It detects exceptions, classifies them by type and severity, and flags them for action. It does not make remediation decisions.

- Address Error → flagged for human intervention — cannot be resolved without correct address data
- Customer Absent → flagged for reroute decision — handled by Agent 3
- Regional Delay → flagged for monitoring and pre-notification

- Failed Attempt → flagged for resolution — handled by Agent 3

Customer notification following any exception is handled by Agent 4. Remediation decisions are handled by Agent 3. Agent 1's only responsibility is detection and classification.

Simulation Results

Shipments	Exceptions Detected	Auto-Executed	Escalated
10	11	5	6

Agent 2 — Courier Performance Monitor

Objective

Score all active couriers across a 30-day rolling window using a composite performance framework. Output a RAG (Red/Amber/Green) rating with a recommended strategic action for each courier.

The Problem It Solves

Individual shipment exceptions (Agent 1) do not tell the full story. A courier might have a bad day — or they might be systematically underperforming. Without a 30-day view, operations teams make courier allocation decisions on recent memory and gut feel rather than data.

Agent 2 provides the strategic layer — separating courier-attributable failures from external factors (weather, customer issues, address errors) to give a clean, fair performance score.

Scoring Framework

- OTD Rate (On-Time Delivery %): Primary metric — weighted 60%
- Courier-Attributable Miss Rate: Only failures directly caused by the courier — not customer absent or address errors
- Attributable Exception Rate: Percentage of shipments with courier-caused exceptions
- Composite Flag: Triggered when OTD < 85% AND courier miss rate > 15% AND exception rate > 20%

Rating Bands

Rating	OTD Threshold	Composite Flag	Action
🟢 Green	≥ 93%	NO	Continue
🟡 Amber	88–92%	NO	Monitor
🔴 Red	< 88%	YES	Suspend

Key Design Decision — Separation of Attributable vs Non-Attributable Misses

A critical design choice was to exclude customer-caused failures (customer absent, address errors) from the courier performance score. This prevents unfair penalisation of couriers for issues outside their control — a distinction that is often missed in real-world courier scoring systems.

Simulation Results — 30-Day Rolling, 300 Shipments

Courier	Shipments	OTD Rate	Courier Miss	Rating	Action
Zipovva Exxpress	80	**69.8%**	80 (100%)	🔴 **Red**	**Suspend**
DPD	77	**90.4%**	0 (0.0%)	🟡 **Amber**	**Monitor**
PostNL	73	**95.0%**	0 (0.0%)	🟢 **Green**	**Continue**

DHL	70	**96.5%**	0 (0.0%)	**Green**	**Continue**

Agent 3 — Exception Resolution Agent

Objective

Given a shipment exception detected by Agent 1, evaluate all available remediation options and recommend the optimal action using a weighted composite score across three dimensions: SLA Recovery, Cost to the Shipper, and Customer Experience.

The Problem It Solves

When a delivery fails, an operations team must decide quickly:

- Re-attempt delivery? — cheap but slow
- Reroute to Parcel Locker or PUDO (Pickup Drop-Off Point)? — fast but customer must collect
- Express Re-ship? — best outcome but expensive
- Full Refund? — last resort, most expensive option for the seller

In real operations this decision falls on a Fleet Manager, Logistics Coordinator or Operations Supervisor — made under pressure, often on gut feel. Agent 3 makes this decision systematically, in seconds, based on a predefined scoring logic.

Remediation Options Library

Option	Trigger Condition	Incremental Cost
Standard Re-attempt	Always available	€2.50
Parcel Locker Reroute	parcel_locker_available = True	€1.50
PUDO Point Reroute	pudo_available = True AND no locker	€1.50
Express Re-ship	express_available = True	€20.00
Proactive Compensation	order_value > €150 OR no options	Full order value lost

Scoring Logic

Each option is scored on three dimensions (0–10 scale each):

Dimension	Weight	What It Measures
SLA Recovery	40%	Can this option physically deliver within the remaining SLA window?
Cost Score	35%	Inverse of incremental cost — lower cost = higher score. Compensation scores 1 (full order value lost).
CX Score	25%	How does the customer feel about this resolution? Express Re-ship scores 9, Self-collect scores lowest.

Composite Score = (SLA × 0.40) + (Cost × 0.35) + (CX × 0.25)

Key Design Decisions

- Express Re-ship SLA score = 0: A new shipment cannot recover the original SLA — it creates a new delivery window. Scoring it as 0 reflects operational reality.
- Compensation SLA score = 0: Issuing a refund does not move the parcel. It is a customer retention tool, not a delivery option.
- PUDO only when no Parcel Locker: Parcel Lockers are 24/7 automated — superior to PUDO which depends on shop opening hours.
- Courier Switch removed: Not operationally realistic for individual shipment in European last mile. Courier performance is a strategic decision handled by Agent 2.
- OTD % not used in scoring: OTD is a 30-day aggregate metric — it has no bearing on which remediation option to choose for an individual shipment.

AUTO-EXECUTE vs ESCALATE Rules

Decision	Trigger Conditions
AUTO-EXECUTE	ALL true: Option cost ≤ €3.00 AND Order value ≤ €150 AND Hours left > 12 AND Not address error
ESCALATE	ANY one: Cost > €3.00 OR Order value > €150 OR Hours left ≤ 12 OR Address error exception

Simulation Results — 10 Shipments

Shipment	Courier	Exception	Urgency	Recommended	Score	Action
SHP-1001	PostNL	Customer Absent	MEDIUM	Parcel Locker	**7.55**	**AUTO**
SHP-1002	Zipovva	Failed Attempt	MEDIUM	PUDO Point	**7.55**	**AUTO**
SHP-1003	DPD	Address Error	MEDIUM	PUDO Point	**7.55**	**ESCALATE**
SHP-1004	DHL	Failed Attempt	MEDIUM	Parcel Locker	**7.55**	**AUTO**
SHP-1005	DPD	Address Error	MEDIUM	Parcel Locker	**7.55**	**ESCALATE**
SHP-1006	PostNL	Address Error	HIGH	Re-attempt	**4.20**	**ESCALATE**
SHP-1007	DPD	Failed Attempt	MEDIUM	Parcel Locker	**7.55**	**AUTO**
SHP-1008	PostNL	Failed Attempt	LOW	Parcel Locker	**7.95**	**AUTO**
SHP-1009	PostNL	Customer Absent	HIGH	Parcel Locker	**6.35**	**ESCALATE**
SHP-1010	Zipovva	Customer Absent	LOW	Parcel Locker	**7.95**	**ESCALATE**
Total: 10 | AUTO-EXECUTE: 5 | ESCALATE: 5 | Avg Composite Score: 7.17 / 10						

Technical Stack & Approach

Component	Detail
Language	Python 3.x — dataclasses, random module, pathlib
Build Tool	Claude Code — agentic AI coding assistant
AI Model	Anthropic Claude (API) — used for Agent 4 rationale generation (upcoming)
Data	Simulated — seeded random generation. All couriers and shipments are fictional.
Couriers	DHL, PostNL, DPD (real), Zipovva Exxpress (fictional for simulation)
Geography	Germany, France, Netherlands
Output	Markdown reports + HTML dashboard screenshots for each agent

---

Key Learnings

- Real ops experience is essential for designing AI decision logic — without it, the scoring tables would be theoretically plausible but operationally wrong (e.g. assuming courier switch is feasible per shipment in EU last mile).
- Human-in-the-loop is not a limitation — it is the correct design for 2026. Full autonomy for high-stakes logistics decisions is neither trusted nor deployed even by large players.
- Simulated data with controlled constraints is more valuable than random data — the minimum-rules approach ensures all edge cases are tested.
- Separating measurement agents (1, 2) from decision agents (3) creates a cleaner, more maintainable architecture.

What's Next

- Agent 4 — Customer Communication Agent: Dynamic AI-generated customer messages based on exception type and resolution. Uses Claude API.
- Agent 5 — Orchestrator: Connects all agents into a single pipeline with a human-in-the-loop approval dashboard.
- Interactive Frontend: Dispatcher dashboard where humans can Approve or Reject Agent 3 recommendations.

Simulation only. All data fictional. Zipovva Exxpress is a fictional courier.

Built with Claude Code | github.com/sarthakshirsatai-lab/ai-logistics-control-tower